Lecture 18: Classes (Chapter 6)

유준수 (Joon Soo Yoo, Ph.D.)

2026년 4월 14일

방산첨단보안연구실 (DAS Lab)
전북대학교 첨단방위산업학과

Chapter 11. Classes

11.1 Object-Oriented Programming (OOP)

객체 지향 프로그래밍(OOP)이란?

현실 세계의 개체(Entity)를 데이터와 기능이 결합된 **객체(Object)**로 모델링하는 프로그래밍 스타일입니다.

- **현실의 예:** 소셜 미디어 계정 (아이디, 팔로워 목록 등 + 메시지 보내기 기능)
- **프로그래밍의 이점:** 코드 재사용성 증대, 대규모 시스템의 체계적 관리 가능

객체의 구성 요소

- **Data:** 객체가 가진 상태나 정보
- **Methods:** 객체가 수행하는 동작

객체 지향 프로그래밍(OOP) 예시: 게임 캐릭터

RPG 게임의 '전사(Warrior)' 캐릭터를 하나의 **객체(Object)**로 모델링해 봅시다.

데이터 (Fields / 상태)

"객체가 무엇을 알고 있는가?"

- 이름 (Name): "아라곤"
- 체력 (HP): 100
- 공격력 (ATK): 25
- 위치 (Pos): (10, 20)

메서드 (Methods / 동작)

"객체가 무엇을 할 수 있는가?"

- `move()`: 위치 좌표 변경
- `attack()`: 상대 체력 감소
- `take_damage()`: 내 체력 감소
- `level_up()`: 공격력 상승

클래스를 쓰는 이유 (1): 캡슐화 - 데이터에 '벽'을 치다

캡슐화는 단순히 코드를 묶는 것이 아니라, 데이터를 보호하는 **접근 권한**의 문제입니다.

- **클래스가 없는 세상 (무질서):** `hp = 100`이라는 변수가 전역 공간에 떠다닙니다. 누구나 실수로 `hp = -999`라고 바꿔서 시스템을 망가뜨릴 수 있습니다.
- **클래스가 있는 세상 (질서):** 데이터를 클래스라는 '벽' 내부에 숨깁니다.

캡슐화의 핵심: 허가된 통로

외부에서 `player.data = 999`라고 직접 건드리는 것을 원천 차단하고, 오직 `player.modify()`처럼 **검증된 메서드**를 통해서만 데이터를 바꿀 수 있게 설계합니다.

클래스를 쓰는 이유 (2): 추상화 - 코드를 '이야기'처럼 만들다

추상화는 복잡함을 숨기고 **객체(Entity) 중심의 직관적인 인터페이스**를 제공합니다.

- **함수형 추상화 (행위 중심):** `calculate_attack(obj_a, obj_b)`
  → "A와 B를 넣고 공격 계산을 수행해라" (기계적인 느낌)
- **클래스형 추상화 (객체 중심):** `joon.attack(soo)`
  → "**준(joon)**이 **수(soo)**를 공격한다" (인간의 언어에 가까운 느낌)

추상화가 주는 가독성의 힘

내부의 수만 줄의 코드가 어떻게 돌아가는지 몰라도, 코드를 읽는 것만으로도 어떤 상황이 벌어지고 있는지 즉시 이해할 수 있게 됩니다.

11.2 Classes and Instances

설계도(Class)와 실체(Instance)

클래스 vs 인스턴스: 설계도와 결과물

수백만 명의 유저 프로필을 관리하려면 각 유저마다 코드를 새로 짜는 대신, 공통된 '틀'이 필요합니다.

- **클래스 (Class):** 객체를 만들기 위한 **설계도(Blueprint)**. 속성(데이터)과 메서드(동작)를 정의합니다.
- **인스턴스 (Instance):** 설계도를 바탕으로 메모리에 실제로 생성된 **개별 객체**입니다.

PEP 8 명명 규칙

클래스 이름은 첫 글자를 대문자로 하는 **PascalCase(CapWords)** 방식을 권장합니다. (예: `GameCharacter`, `FlightTicket`)

__init__()와 self: 객체의 탄생

신규 객체가 생성될 때 파이썬이 가장 먼저 호출하는 특별한 메서드입니다.

Warrior 클래스 정의

```
class Warrior:
  def __init__(self, name, hp):
    self.name = name   # 인스턴스 속성 초기화
    self.hp = hp

p1 = Warrior("아라곤", 100)   # 인스턴스 생성
```

- **self의 정체:** "나 자신"을 가리킵니다. 메서드를 호출한 **그 특정 인스턴스**를 지칭하여 속성에 접근할 수 있게 합니다.
- **속성(Attribute):** `self.name`처럼 인스턴스에 소속된 변수를 말합니다.

왜 이름이 하필 `__init__`이고 `self`인가요?

처음 보면 당황스럽지만, 파이썬의 핵심 약속(Protocol)을 이해하면 아주 쉬워집니다.

1. `__init__`: 파이썬이 관리하는 '준비' 메서드

- **이름의 의미:** 'Initialize(초기화하다)'의 줄임말
- **던더(__):** 앞뒤의 밑줄 두 개는 "이건 파이썬 시스템이 알아서 관리할 테니, 너는 규칙만 지켜!" 라는 뜻

2. `self.name = name`: "외부 배달물을 내 사물함에 넣기"

가장 많이 질문하는 부분! 왜 똑같은 이름을 두 번 쓸까?

- **오른쪽** `name`: 외부에서 전달받은 **'배달물'** (임시 변수)
- **왼쪽** `self.name`: 객체 내부에 평생 간직할 **'개인 사물함'** (속성)
- **즉,** "밖에서 온 '아라곤'이라는 배달물을 내(`self`) 이름(`name`) 칸에 저장해!"라는 뜻

인스턴스 속성 vs 클래스 속성

값이 각자 다른가, 혹은 모두가 공유하는가의 차이입니다.

속성의 종류 비교

구분	인스턴스 속성 (Instance)	클래스 속성 (Class)
공유 여부	각 인스턴스마다 **고유함**	모든 인스턴스가 **공유함**
접근 방법	`self.attribute`	`ClassName.attribute`
예시 (게임)	캐릭터 이름, 현재 HP	게임 타이틀, 서버 위치

11.2 실전 예시: 속성(Attribute)의 차이

Warrior 클래스 설계

```
class Warrior:
    game_title = "반지의 제왕"  # 클래스 속성: 모든 전사가 공유
    server = "KR-Seoul"      # 클래스 속성: 모든 전사가 공유

    def __init__(self, name, hp):
        self.name = name     # 인스턴스 속성: 캐릭터마다 다름
        self.hp = hp         # 인스턴스 속성: 캐릭터마다 다름

p1 = Warrior("아라곤", 100)
p2 = Warrior("보로미르", 80)
```

Task 1: 게임 캐릭터 관리 (Class vs Instance)

클래스 속성과 인스턴스 속성의 차이를 이해하며 캐릭터를 설계해 봅시다.

Mission: `Warrior` 클래스를 설계하세요.

1. **인스턴스 속성 (개인 정보):** `nickname, job, hp`를 `__init__`에서 설정하세요.
2. **클래스 속성 (공통 규칙):** 모든 캐릭터가 소속된 `server = "KOR_Main"`을 클래스 바로 아래에 선언하세요.
3. **객체 생성:** 'kim' 전사와 'lee' 전사 객체를 각각 생성하세요.
4. **출력:** 각 객체의 정보와 함께, 서버 이름이 동일하게 출력되는지 확인하세요.

핵심 체크포인트

- `self.nickname`: "내 닉네임은 뭐야!" (인스턴스마다 다름)
- `Warrior.server`: "우리 서버는 여기야!" (모든 인스턴스가 공유)

11.3 Instance Methods

객체의 탄생 설정과 동작 정의

더 강력한 __init__(): 기본값 활용

객체를 만들 때 모든 정보를 다 줄 수도 있지만, 자주 쓰이는 값은 **기본값(Default Value)**으로 설정해둘 수 있습니다.

Warrior 클래스의 진화

```
class Warrior:
    def __init__(self, name, hp=100, level=1):
        self.name = name
        self.hp = hp
        self.level = level

# 다양한 방식으로 객체 생성 가능
p1 = Warrior("아라곤")            # hp=100, level=1 (기본값)
p2 = Warrior("간달프", hp=80)     # level=1 (기본값)
p3 = Warrior("레골라스", 90, 5)   # 모든 값 직접 지정
```

중요 규칙

항상 `self`가 첫 번째 매개변수여야 하며, 기본값이 있는 매개변수는 반드시 뒤쪽에 배치해야 합니다.

메서드(Method): 객체의 능동적인 "동사"

클래스가 '명사(사물)'라면, 메서드는 그 사물이 할 수 있는 **'동사(행동)'**입니다.

- **속성 (Attribute):** 객체가 무엇을 **아는가?** (이름, 체력, 공격력)
- **메서드 (Method):** 객체가 무엇을 **하는가?** (걷다, 공격하다, 회복하다)

비유

붕어빵을 예로 들면 '팥'이나 '슈크림'은 **속성**이지만, 이 붕어빵이 김을 모락모락 내뿜거나 맛있는 냄새를 풍기는 것은 **메서드**라고 할 수 있음

행동의 재료: 객체의 데이터(self) 활용

메서드는 단순히 코드를 실행하는 것이 아니라, 객체 내부에 저장된 **자신의 정보**를 꺼내어 사용합니다.

내 정보를 활용한 동작

```
def take_damage(self, power):
    # '내 체력(self.hp)'을 재료로 삼아 피해를 계산합니다.
    self.hp -= power
    print(f"윽! 내({self.name}) 체력이 {self.hp} 남았어.")
```

- **데이터 접근:** `self.hp`를 확인해 현재 상태를 파악합니다.
- **데이터 수정:** 행동의 결과로 `self.hp`를 직접 업데이트합니다.

객체 지향의 꽃: "행동을 요청하기"

메서드를 사용하면 코드가 기계적인 계산식이 아니라, 객체들 사이의 **직관적인 이야기**가 됩니다.

함수 중심 (기계적)

`calculate_hp(joon, soo, 10)`
→ "준과 수의 데이터를 넣고 계산해라"

객체 중심 (직관적)

`joon.attack(soo)`
→ "**준**이 **수**를 **공격**한다"

결론: 메서드는 복잡한 내부 로직을 숨기고, 사용자가 객체에게 **"이거 해!"**라고 쉽게 명령할 수 있게 해줍니다.

Task 2: 자판기 시스템 (Vending Machine)

Mission: 아래 기능을 가진 `VendingMachine` 클래스를 만드세요.

- **속성:** `count` (현재 재고), `max` (최대 용량)
- **메서드:**
  1. `__init__(num)`: `count`와 `max`를 `num`으로 초기화
  2. `refill()`: 재고를 최대로 채우고 "Refilled" 출력
  3. `sell(order)`: 재고에서 `order`만큼 차감하고 "Sold: [n]" 출력
  4. `print_stock()`: "Current stock: [n]" 출력

실행 시나리오

`m = VendingMachine(100)` → `m.sell(25)` → `m.refill()`

결과: Stock: 100 → Sold: 25 → Stock: 75 → Refilled → Stock: 100

Any Questions?